as matches?, you can write the expectation as be_a_match. In this case RSpec will look for match? and then form matches? if it can't find match?.

Similarly, if the predicate method starts with has, RSpec allows your matcher to start with have for readability (so your tests don't look like they've been written by LOLcats); RSpec allows expect(actual).to have_key(:id) rather than expect(actual).to has_key(:id).

RSpec also allows you to chain multiple matchers using and and or, as in expect(actual).to include("a").and match(/.*3.*/), or expect(actual).to eq(3).or eq(5).

You can also pass matchers as arguments to other matchers, or compose matchers to handle an entire data structure using match. The following two snippets are equivalent:

```
expect(actual[0]).to eq(5)
expect(actual[1]).to eq(7)
```

```
expect(actual).to match([an_object_eq_to(5), an_object_eq_to(7)])
```

In the bottom snippet, the elements of the array actual are individually matched against an_object_eq_to(5) and an_object_eq_to(7), which are aliases to eq(5) and eq(7). Most of the built-in matchers have aliases to make them more natural-language-like when used as arguments. (You can see a full list online.[3]) The all matcher, which looks like expect(actual).to all(be_truthy), takes the matcher argument (in this case be_truthy) and applies it to each element in the object being matched (in this case, actual, which should be enumerable).

I love this language inflection because I feel like it allows me to write tests that are succinct and clear. Others find the internal complexity of these features to be too high a price to pay for the natural-language tests. (Some people also really don't like the natural-language syntax.)

Back on Task

What remains of your definition of done? is the distinction between complete and incomplete tasks. Let's start with that, with a test for Task:

basics/03/spec/models/task_spec.rb

```
require "rails_helper"

RSpec.describe Task do
  let(:task) { Task.new }

  it "does not have a new task as complete" do
    expect(task).not_to be_complete
  end
```

3. https://relishapp.com/rspec/rspec-expectations/v/3-7/docs/built-in-matchers